

ICS-Lab1: DataLab实验报告

吴世强 24300810019@m.fudan.edu.cn

Part 0. 提交注意事项

截止时间：10月9日 23:59。逾期将扣除部分分数。

内容要求

你需要提交至少两份文件，包含你的 `bits.c` 和一份实验报告。

实验报告应该包含以下内容：

- 实验标题，你的姓名，学号。
- 你在终端中执行 `./check_ops.py bits.c` 后的截图。
- 你在终端中执行 `./btest` 后的截图。
- 描述你实现每个函数的思路。 `bits.c` 中不要求给自己的代码写注释（写了也无妨）
- 如果有，请务必在报告中列出引用的内容以及参考的资料。
- 对本实验的感受（可选）。
- 对助教们的建议（可选）。

测试

除了 `bits.c`，你不应该编辑任何其余文件。

编译

每次修改 `bits.c` 后重新编译：

代码块

```
1 make clean
2 make all
```

检查运算符和操作数

代码块

```
1 ./check_ops.py bits.c
```

输出会列出每道题的“当前操作数/最大操作数”。最后出现以下内容才表示规则检查通过：

代码块

```
1 All 19 functions passed operator checks.
```

Part 1. 基础知识复习

整数表示

1. 无符号整数 (Unsigned)

- **表示**：所有位都用来表示数值大小。
- **范围**：对于 w 位，范围是 0 到 $2^w - 1$ 。
- **编码**：二进制直接对应数值。

2. 补码 (Two 's Complement)

这是**有符号整数**的标准表示法。

- **最高位是符号位**：0 表示非负，1 表示负数。
- **范围**： -2^{w-1} 到 $2^{w-1} - 1$ 。
- **编码公式**：

$$B2T_w(\vec{x}) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

- **重要特性**：
 - 零有唯一表示：全 0。
 - 最小值 $TMin = -2^{w-1}$ ，最大值 $TMax = 2^{w-1} - 1$ 。
 - 不对称性： $|TMin| = TMax + 1$ 。
 - 取反加一： $-x = \sim x + 1$ （补码的相反数）。

3. 有符号与无符号之间的转换

- **位模式不变，解释方式改变**。
- **转换公式**（ w 位）：
 - 无符号 $\rightarrow$ 补码：若 $u \geq 2^{w-1}$ ，则 $u - 2^w$ ；否则不变。
 - 补码 $\rightarrow$ 无符号：若 $x < 0$ ，则 $x + 2^w$ ；否则不变。
- **C 语言中的隐式转换**：有符号与无符号混合运算时，**有符号会被隐式转换为无符号**，容易导致 bug（如 `-1 > 0U` 为真）。

4. 扩展与截断

- 零扩展：无符号数扩展时高位补 0。
- 符号扩展：补码扩展时高位补符号位。
- 截断：直接丢弃高位，可能导致数值改变（取模运算）。

5. 整数运算

- 无符号加法：模 2^w 运算，溢出则减去 2^w 。
- 补码加法：先按无符号加，再按补码解释。溢出分为正溢出和负溢出。
- 乘法：结果截断到 w 位，等价于模 2^w 。
- 除以 2 的幂：
 - 无符号：逻辑右移。逻辑右移高位补 0
 - 补码：算术右移，但负数需要加偏置 $(2^k - 1)$ 再移位，以实现向零舍入。算术右移高位补符号位

核心区别：右移腾出的高位补什么

逻辑右移 高位补 0（按无符号数理解）； 算术右移 高位补 符号位（原最高位），负数右移后仍保持为负。

例 1：正数 0x12345678 >> 4（最高位为 0）—— 逻辑、算术结果相同

原值	1	2	3	4	5	6	7	8	0x12345678
逻辑右移	0	1	2	3	4	5	6	7	0x01234567
算术右移	0	1	2	3	4	5	6	7	0x01234567（相同）

例 2：负数 0x87654321 >> 4（最高位为 1）—— 结果分道扬镳

原值	8	7	6	5	4	3	2	1	0x87654321
逻辑右移	0	8	7	6	5	4	3	2	0x08765432（变成正数）
算术右移	F	8	7	6	5	4	3	2	0xF8765432（仍是负数）

二、浮点数表示（IEEE 754 标准）

1. IEEE 754 浮点格式

一个浮点数由三部分组成：

部分	符号位 s	阶码 exp	尾数 frac
----	-------	--------	---------

单精度	1 位	8 位	23 位
双精度	1 位	11 位	52 位

- 符号位 s : 0 正, 1 负。
- 阶码 exp : 用移码表示 (偏置值 $Bias = 2^{k-1} - 1$, 单精度 127, 双精度 1023)。
- 尾数 $frac$: 表示小数部分, 隐含前导 1 (规格化数)。

2. 数值分类 (以单精度为例)

根据 exp 和 $frac$ 的不同, 分为四类:

1. 规格化数 (Normalized)

- 条件: $exp \neq 0$ 且 $exp \neq 255$
- 阶码真值: $E = exp - Bias$
- 尾数真值: $M = 1 + frac$ (隐含 1)
- 值: $V = (-1)^s \times M \times 2^E$

2. 非规格化数 (Denormalized)

- 条件: $exp = 0$
- 阶码真值: $E = 1 - Bias$ (注意不是 $0 - Bias$)
- 尾数真值: $M = frac$ (没有隐含 1)
- 作用: 表示 0 和非常接近 0 的数, 提供渐进下溢。

3. 无穷大 (Infinity)

- 条件: $exp = 255$ 且 $frac = 0$
- 表示 $\pm\infty$, 用于溢出或除以零。

4. NaN (Not a Number)

- 条件: $exp = 255$ 且 $frac \neq 0$
- 表示非法运算结果, 如 $\sqrt{-1}$ 、 $\infty - \infty$ 。

4. 浮点数的舍入

- 默认舍入模式: 向偶数舍入 (Round to Even), 也叫最近舍入。
- 中间值时, 舍入到最近的偶数 (最低有效位为 0)。

Part 2. 每个函数的实现思路

0. 实现要求

整数题

- 只能改每个函数里的 `return`，用局部 `int` 变量和表达式实现。
- 常量只能用 `0` 到 `255` (`0xFF`)，不能写 `0xffffffff` 这种大常量。
- 允许的运算符只有 `! ~ & ^ | + << >>`。
- 禁止 `if`、循环、宏、额外函数、函数调用、`&&` `||` `-` `?:`、类型转换，以及 `int` 以外的类型（数组、结构体都不行）。
- 机器按 32 位补码、算术右移来理解；移位量必须在 `0` 到 `31` 之间。

浮点题

- 规则松一些：可以用循环、条件、`int` 和 `unsigned`、任意整数常量，以及普通算术、逻辑、比较运算。
- 仍然禁止宏、额外函数、函数调用、类型转换，也不能用浮点类型、浮点运算或浮点常量。

检查

- 每个函数的运算次数有上限，以本文件函数头注释为准。
- 用 `dlc` 查是否符合规则，用 `btest` 查正确性，用 BDD checker 做形式化验证。

1. signMask

功能：返回一个只将最高位置 1 的掩码 (`0x80000000`)。

允许的运算符： `! ~ & ^ | + << >>`

最大运算次数：2 | 难度评级：1

代码块

```
1  int signMask(void) {
2      return 1 << 31;
3  }
```

💡 显然易证：把 1 左移 31 位，即 `1 << 31`。1 从最低位被一路搬到最高位，其余位补 0，正好得到 `0x80000000`——也就是“最高位置 1、其余全 0”的掩码。

2. bitXor

功能：仅使用 `~` 和 `&` 实现按位异或 x^y 。示例：`bitXor(4, 5) = 1`，`bitXor(7, 7) = 0`。

允许的运算符： `~ &`

最大运算次数：8 | 难度评级：2

```

1 代码块 int bitXor(int x, int y) {
2      int x_ny = x & ~y;
3      int nx_y = ~x & y;
4      return ~(~x_ny & ~nx_y);
5  }

```

 由德摩根律知：

$$a | b == \sim(\sim a \& \sim b)$$

不难想到这道题目可以写成 $(x \& \sim y) | (\sim x \& y)$

代入德摩根律知道 $\sim(\sim(x \& \sim y) \& \sim(\sim x \& y))$

3. negativePart

功能：若 $x < 0$ 则返回 $-x$ ，否则返回 0。示例：negativePart(-10) = 10，negativePart(5) = 0。

允许的运算符： `! ~ & ^ | + << >>`

最大运算次数：6 | 难度评级：3

代码块

```

1  int negativePart(int x){
2      int mask = x >> 31;
3      int neg = ~x + 1;
4      return mask & neg;
5  }

```

 思路：x 为负数时符号位是 1，非负数时符号位是 0。把 x 算术右移 31 位：负数得到 0xFFFFFFFF（全 1），非负数得到 0x00000000（全 0），这个结果正好可以当掩码用。

补码中 $-x = \sim x + 1$ （取反加一），所以先算出 $neg = \sim x + 1$ ，再与掩码按位与：
 $x < 0$ 时掩码全 1，结果就是 $-x$ ； $x \geq 0$ 时掩码全 0，结果归 0，正好满足“负数返回 $-x$ ，否则返回 0”。

4. copyByteWithin

功能：把 x 的第 src 个字节复制到第 dst 个字节，其余字节保持不变。

说明：字节从 0（最低有效字节）编号到 3（最高有效字节）；可假设 $0 \leq src \leq 3$ ， $0 \leq dst \leq 3$ 。

示例：copyByteWithin(0x11223344, 0, 2) = 0x11443344。

允许的运算符： `! ~ & ^ | + << >>`

最大运算次数：12 | 难度评级：4

代码块

```
1  int copyByteWithin(int x, int src, int dst) {
2      // get the byte that is from source
3      int byte = (x >> (src << 3)) & 0xFF;
4      // clean the destination source from the number
5      int mask = ~(0xFF << (dst << 3));
6      int masked_x = mask & x;
7      return masked_x | (byte << (dst << 3));
8  }
```

💡 先取出 src 字节：第 k 个字节对应位偏移 $k*8$ （即 $k \ll 3$ ），所以 $(x \gg (src \ll 3)) \& 0xFF$ 得到第 src 个字节的值。

再清空目标字节：构造掩码 $\sim(0xFF \ll (dst \ll 3))$ ，它只把第 dst 个字节对应的 8 位清零，与 x 相与后其余字节原样保留。

最后把取出的字节放回 dst 位置： $(byte \ll (dst \ll 3)) \mid masked_x$ ，完成复制。

5. logicalShift

功能：将 x 逻辑右移 n 位（高位补 0）。

说明：可假设 $0 \leq n \leq 31$ 。示例：`logicalShift(0x87654321, 4) = 0x08765432`。

允许的运算符：`! ~ & ^ | + << >>`

最大运算次数：20 | 难度评级：4

代码块

```
1  int logicalShift(int x, int n) {
2      // 最高的 n+1 位全是 1
3      int mask = ~(((1 << 31) >> n) << 1);
4      return (x >> n) & mask;
5  }
```

💡 C 里的 `x >> n` 是算术右移，高 n 位会补符号位（负数补 1），所以需要构造掩码把高位填进去的符号位副本清掉。

`1 << 31` 得到只有最高位为 1 的 `0x80000000`；`(1 << 31) >> n` 算术右移 n 位后符号位 1 被复制，得到"最高 n+1 位全是 1、其余全 0"的中间值

再 `<< 1` 把最低的 1 移出，变成"最高 n 位全是 1"；取反后 mask 即为"高 n 位为 0、低 32-n 位为 1"，正好把算术右移填进高位的符号位副本全部清零。

最后 `(x >> n) & mask` 就是逻辑右移的结果。

6. swapNibblePairs

功能：交换 x 中每个字节的低 4 位与高 4 位。示例：swapNibblePairs(0xAB) = 0xBA。

允许的运算符： `! ~ & ^ | + << >>`

最大运算次数：18 | 难度评级：4

代码块

```
1  int swapNibblePairs(int x) {
2      int mask = (0x0F << 24) | (0x0F << 16) | (0x0F << 8) | 0x0F;
3      return ((x >> 4) & mask) | ((x & mask) << 4);
4  }
```



这里面我们构造的mask是一个形状如 0x0F0F0F0F 的掩码

`((x >> 4) & mask)` 我们先把这个 x 整体右移 4 位，每个 byte 里面再通过掩码，只取最后 4 位，这样我们就得到了一个把每个 byte 前 4 位移到后 4 位的一个数字

`((x & mask) << 4)` 我们先通过掩码，得到 x 里面每个 byte 前 4 位的一个数字，再整体左移 4 位

两者进行与操作，我们得到了每 byte 的前 4 位和后 4 位进行交换的一个数字

7. secondLowestZeroBit

功能：返回一个掩码，标记 x 中从低位起第二个 0 位的位置。

示例：secondLowestZeroBit(0xFFFFFFFF) = 0x4, secondLowestZeroBit(0x7FFFFFFF) = 0, secondLowestZeroBit(-1) = 0。

允许的运算符： `! ~ & ^ | + << >>`

最大运算次数：8 | 难度评级：4

代码块

```
1  int secondLowestZeroBit(int x) {
2      /* x + 1 负责把第一个0之后的所有1全部清成0，然后我们这个第一个0变成1
3      注意到，如果我们对于 x 来进行 ~ 操作，那么第一个0变成1，前面所有位都和 x 相反，也就和
       x + 1 相反
4      然后再进行 & 操作，我们就得到了只有最低一位是0的这个掩码
```

```

5    */
6    int lowest_zero_mask = (~x) & (x + 1);
7    /* 然后我们把这个最低位的0变成1*/
8    int x1 = x ^ lowest_zero_mask;
9    /* 重复一次取掩码操作，得到第二个最低位的0的掩码*/
10   return (~x1) & (x1 + 1);
11   }

```

💡 函数的目标是返回一个掩码：它只在 x 中“从低位起第二个 0 位”的位置为 1，其余位全为 0。

核心技巧是先掌握如何定位“最低位的 0”： $(\sim x) \& (x + 1)$ 。

看 $x + 1$ ：从最低位开始，遇到第一个 0 之前若有连续的一串 1，它们会不断进位——第一个 0 变成 1，它之后的所有 1 全部被清成 0；再看 $\sim x$ ： x 中的第一个 0 在这里变成 1。两者按位与后，只有“最低位 0”所在的那一位同时在两边都是 1，其余位至少有一边是 0，于是得到仅该位为 1 的掩码 `lowest_zero_mask`。

接着 $x1 = x \oplus \text{lowest_zero_mask}$ ：**把这个最低位的 0 翻成 1，相当于把它“消除”**，于是 $x1$ 中从低位起的第一个 0，正好就是原来 x 的第二个 0。

最后对 $x1$ 重复一次取掩码操作 $(\sim x1) \& (x1 + 1)$ ，得到的掩码标记的正是 x 中第二个最低位的 0，直接返回即可。

8. oddParity

功能：返回 x 的奇校验位：当 x 的二进制表示中 1 的个数为偶数时返回 1，否则返回 0。示例：
`oddParity(5) = 1`，`oddParity(7) = 0`。

允许的运算符：`! ~ & ^ | + << >>`

最大运算次数：56 | 难度评级：5

代码块

```

1  int oddParity(int x) {
2      /* 我们把 x 分成两部分，每部分16位，然后我们分别计算每部分的奇偶性*/
3      int up_16bit = (x & ((0xFF << 24) | (0xFF << 16))) >> 16;
4      int down_16bit = x & ((0xFF << 8) | 0xFF);
5      int half = up_16bit ^ down_16bit;
6
7      /* 我们把 half 分成两部分，每部分8位，然后我们分别计算每部分的奇偶性*/
8      int up_8bit = (half & ((0xFF << 8))) >> 8;
9      int down_8bit = half & (0xFF);
10     int quarter = up_8bit ^ down_8bit;
11
12     /* 我们把 quarter 分成两部分，每部分4位，然后我们分别计算每部分的奇偶性*/

```

```

13     int up_4bit = (quarter & (0xF0)) >> 4;
14     int down_4bit = quarter & 0xF;
15     int eighth = up_4bit ^ down_4bit;
16
17     /* 我们把 eighth 分成两部分，每部分2位，然后我们分别计算每部分的奇偶性*/
18     int up_2bit = (eighth & (0xC)) >> 2;
19     int down_2bit = eighth & 0x3;
20     int sixteenth = up_2bit ^ down_2bit;
21
22     /* 我们把 sixteenth 分成两部分，每部分1位，然后我们分别计算每部分的奇偶性*/
23     int up_1bit = (sixteenth & (0x2)) >> 1;
24     int down_1bit = sixteenth & 0x1;
25     int thirty_second = up_1bit ^ down_1bit;
26
27     return (thirty_second & 0x1) ^ 1;
28 }

```



我们注意到一个有趣的事实：

对一个二进制数，他的前半部分和后半部分，分别取异或，得到的这个“折半”的新的二进制数，和原来的那个二进制数的奇偶性是一样的。如果一个位置上面，两个数都是1，那么异或之后，相当于同时少了两个1，这不影响这个奇偶性。

因此我们可以不断的进行二分，重复以上的操作，来的到这个奇偶性不变，但是长度不断缩短的这个二进制数，等长度已经缩短到1的时候，我们就把这个数来进行奇偶校验。

9. rotateRightBits

功能：将 x 循环右移 n 位。

说明：可假设 $n \geq 0$ 。示例：rotateRightBits(0x12345678, 8) = 0x78123456。

允许的运算符： `! ~ & ^ | + << >>`

最大运算次数：16 | 难度评级：5

代码块

```

1  int rotateRightBits(int x, int n) {
2      int thirtytwo_minus_n = 0x20 + (~n + 1);
3      /*最低的 n 位都是 1 的mask*/
4      int mask = (1 << n) + ~0;
5      /*最高的 n 位是 0 的mask*/
6      int high_mask = ~(((1 << 31) >> n) << 1);
7
8      int masked_x = mask & x;
9

```

```
10     return ((x >> n) & high_mask) | (masked_x << thirtytwo_minus_n);
11 }
```

💡 循环右移可以拆成“**右移部分 + 移出的低位搬回高位**”两半来看。

先取低 n 位：mask = (1 << n) + ~0 得到“低 n 位全 1”的掩码（1 左移 n 位后第 n 位为 1，加上全 1 的 ~0 即 -1，低 n 位全部变成 1），masked_x = mask & x 就把 x 的低 n 位（即将移出循环的部分）保存下来。

再处理右移部分：C 的 x >> n 是算术右移，高 n 位会补符号位（负数补 1），需要构造掩码把它清掉——high_mask 的构造与 logicalShift 完全一致：1 << 31 右移 n 位得到“最高 n+1 位为 1”，再左移 1 位并取反，得到“高 n 位为 0、低 32-n 位为 1”的掩码，与 (x >> n) 相与后，补进来的符号位副本被全部清零。

最后拼接：thirtytwo_minus_n = 0x20 + (~n + 1) 计算的是 32 - n（题目不允许减法运算符，用补码 ~n + 1 表示 -n）；**(masked_x << thirtytwo_minus_n) 把低 n 位移回最高位，再与右移结果按位或，即完成循环右移。**

10. roundEvenPow2

功能：将非负数 x 四舍五入到 2^n 的最近倍数；若 x 恰好位于两个倍数的正中间，则选择除以 2^n 后商为偶数的那个倍数。

说明：可假设 $0 \leq x \leq 0x3fffffff$ ， $1 \leq n \leq 16$ 。示例：roundEvenPow2(10, 2) = 8，roundEvenPow2(14, 2) = 16。

允许的运算符： ! ~ & ^ | + << >>

最大运算次数：24 | 难度评级：5

代码块

```
1  int roundEvenPow2(int x, int n) {
2      int mask = (1 << n) + ~0;
3      int r = x & mask;
4      int half = 1 << (n + ~0);
5      int q = x >> n;
6      int tie_up = (!(r ^ half)) & (q & 1);
7      int bias = half + ~0 + tie_up;
8      return ((x + bias) >> n) << n;
9  }
```

💡 把 x 拆成商和余数：q = x >> n 是向下取整的商，r = x & ((1 << n) + ~0) 是低 n 位余数，half = 1 << (n - 1) 是一半。

舍入规则：**r 小于 half 时向下（保持 q）**；**r 大于 half 时向上（q + 1）**；r 恰好等于 half（x 正好位于两个倍数的正中间）时按“向偶”处理——只有 q 为奇数才进位，q 为偶数保持不变。

实现上构造一个偏置 $\text{bias} = (\text{half} - 1) + \text{tie_up}$ ，其中 **tie_up 仅在“r == half 且 q 为奇数”时为 1**，即 $\text{tie_up} = !(r \wedge \text{half}) \& (q \& 1)$ ；把 bias 加到 x 上再统一截断： $(x + \text{bias}) \gg n \ll n$ ，一次算完。验证： $\text{roundEvenPow2}(10, 2)$ 中 $q = 2$ 、 $r = \text{half}$ 且 q 为偶数，不进位得 8； $\text{roundEvenPow2}(14, 2)$ 中 $q = 3$ 为奇数，进位得 16。

11. midpointTowardFirst

功能：返回 $(x+y)/2$ 的精确数学中点且不溢出；若中点恰好位于两个整数正中间，则选择更靠近第一个参数 x 的那个整数。示例： $\text{midpointTowardFirst}(4, 7) = 5$ ， $\text{midpointTowardFirst}(7, 4) = 6$ 。

允许的运算符：`! ~ & ^ | + << >>`

最大运算次数：32 | 难度评级：5

代码块

```
1  int midpointTowardFirst(int x, int y) {
2      int floor_mid = (x >> 1) + (y >> 1) + ((x & y) & 1);
3      int odd_sum = (x ^ y) & 1;
4      int sign_x = (x >> 31) & 1;
5      int sign_y = (y >> 31) & 1;
6      int diff_sign = sign_x ^ sign_y;
7      int sub = y + (~x + 1);
8      int sub_sign = (sub >> 31) & 1;
9      int x_gt_y = (diff_sign & !sign_x) | ((!diff_sign) & sub_sign);
10     return floor_mid + (odd_sum & x_gt_y);
11 }
```

💡 直接算 $x + y$ 会溢出，所以把中点拆开。算术右移是向下取整， $(x \gg 1) + (y \gg 1)$ 先得到两个商；两个数都是奇数时，各自丢掉的最低位还要补回一个进位 $(x \& y \& 1)$ 。这样得到的是 $\text{floor}((x+y)/2)$ 。

x、y 奇偶不同时，和是奇数，中点正好落在两个整数中间。这时如果 $x > y$ ，就再加 1，结果靠近较大的那个 x；否则保持向下取整，靠近较小的 x。

比较大小时要避免溢出。符号不同时，x 非负而 y 为负才有 $x > y$ ，不必做减法。符号相同时， $y - x$ 不会溢出，差为负就说明 x 更大。INT_MIN 和 INT_MAX 也落在这两种情况里。

12. isBetweenEitherOrder

功能：当 x 位于以 a 、 b 为端点的闭区间内时返回 1；端点 a 、 b 的给出顺序不限。示例：
`isBetweenEitherOrder(5, 8, 3) = 1`。

允许的运算符： `! ~ & ^ | + << >>`

最大运算次数：48 | 难度评级：7

代码块

```
1  int isBetweenEitherOrder(int x, int a, int b) {
2      int sx = x >> 31;
3      int sa = a >> 31;
4      int sb = b >> 31;
5      int sxa = sx ^ sa;
6      int nsxa = ~sxa;
7      int nsx = ~sx;
8      int dxa = x + (~a + 1);
9      int x_ge_a = (sxa & nsx) | (nsxa & ~(dxa >> 31));
10     int a_ge_x = (sxa & sx) | (nsxa & ((dxa + ~0) >> 31));
11     int sxb = sx ^ sb;
12     int nsxb = ~sxb;
13     int dxb = x + (~b + 1);
14     int x_ge_b = (sxb & nsx) | (nsxb & ~(dxb >> 31));
15     int b_ge_x = (sxb & sx) | (nsxb & ((dxb + ~0) >> 31));
16     return !((x_ge_a & b_ge_x) | (x_ge_b & a_ge_x));
17 }
```

 x 在区间内，就是 $(x \geq a \text{ 且 } b \geq x)$ ，或者把 a 、 b 对调之后同样成立。
直接相减会溢出，所以先看符号。符号不同时，非负数一定不小于负数，用符号位就能判断，不必使用那个可能溢出的差值。符号相同时， $x - a$ 一定落在 `int` 范围内：差值非负表示 $x \geq a$ ；把差值减 1 再看符号，0 也会被算进 $a \geq x$ 。

四组关系都用全 0 或全 1 的掩码表示，两种顺序用 `&` 和 `|` 拼起来，最后 `!!` 收成 0 或 1。

13. mul5Sat

功能：返回 $x*5$ ；若 $x*5$ 溢出，则将结果相应饱和为 `INT_MAX` (`0x7fffffff`) 或 `INT_MIN` (`0x80000000`)。示例：`mul5Sat(1) = 0x5`，`mul5Sat(0x40000000) = 0x7fffffff`。

允许的运算符： `! ~ & ^ | + << >>`

最大运算次数：30 | 难度评级：7

代码块

```
1  int mul5Sat(int x) {
```

```

2    int sign = x >> 31;
3    int shift_ov = (x >> 29) ^ sign;
4    int prod = (x << 2) + x;
5    int add_ov = (prod ^ x) >> 31;
6    int ov = (!(shift_ov) | !(add_ov));
7    int sat = sign ^ ~(1 << 31);
8    int ov_mask = ~ov + 1;
9    return (ov_mask & sat) | (~ov_mask & prod);
10 }

```

💡 $5x = (x \ll 2) + x$ 。左移 2 位不溢出时，最高 3 位必须相同，也就是 $x \gg 29$ 等于符号位扩展出来的 0 或全 1。加法再溢出时，prod 的符号会和 x 不同。

两种溢出都要饱和。饱和值写成 $\text{sign} \wedge \sim(1 \ll 31)$ ：x 非负时是 INT_MAX，x 为负时是 INT_MIN。溢出标志先收成 0 或 1，再变成全 0 或全 1 的掩码，在饱和值和正常乘积之间选择。

14. classifyAdd3

功能：判断 $x+y+z$ 的精确数学和：和大于 INT_MAX 返回 1，小于 INT_MIN 返回 -1，否则返回 0。不允许使用更宽的整数类型。

允许的运算符： `! ~ & ^ | + << >>`

最大运算次数：52 | 难度评级：7

代码块

```

1    int classifyAdd3(int x, int y, int z) {
2        int s1 = x + y;
3        int s2 = s1 + z;
4        int sx = x >> 31;
5        int sy = y >> 31;
6        int sz = z >> 31;
7        int ss1 = s1 >> 31;
8        int ss2 = s2 >> 31;
9        int pos = (~sx & ~sy & ss1) | (~ss1 & ~sz & ss2);
10       int neg = (sx & sy & ~ss1) | (ss1 & sz & ~ss2);
11       pos = (pos >> 31) & 1;
12       neg = (neg >> 31) & 1;
13       return pos + (~neg + 1);
14   }

```

💡 分两步相加，每一步用符号判断溢出：两个非负数相加得到负数，是正溢出；两个负数相加得到非负数，是负溢出。

第一步正溢出之后，s1 的位模式已经变成负数，第二步只可能再出现负溢出，不会连续两次同方向溢出。两次溢出方向相反时，各自多出来或少掉的 2^{32} 正好抵消，数学和仍在 int 范围内。只有正溢出返回 1，只有负溢出返回 -1，否则返回 0。pos + (~neg + 1) 就是这个结果。

15. floatScaleThreeHalves

功能：返回浮点参数 f 的 $f*3/2$ 的位级等价表示。参数与结果均以 unsigned int 传递，但应理解为单精度浮点数的位级表示。采用向最近偶数舍入；保留 +0 与 -0 的符号；参数为 NaN 时返回参数本身。

允许的运算符：任意整数/无符号运算（含 ||、&&），也可使用 if、while

最大运算次数：60 | 难度评级：7

代码块

```
1  unsigned floatScaleThreeHalves(unsigned uf) {
2      unsigned sign = uf & 0x80000000u;
3      unsigned exp = (uf >> 23) & 0xffu;
4      unsigned frac = uf & 0x7ffffffu;
5      unsigned sig;
6      unsigned kept;
7      unsigned round;
8
9      if (exp == 255u)
10         return uf;
11     if (!(uf & 0x7fffffffu))
12         return uf;
13
14     if (exp == 0u) {
15         sig = frac * 3u;
16         kept = sig >> 1;
17         if ((sig & 1u) && (kept & 1u))
18             kept = kept + 1u;
19         return sign | kept;
20     }
21
22     sig = (frac | 0x800000u) * 3u;
23     if (sig & 0x2000000u) {
24         kept = sig >> 2;
25         round = sig & 3u;
26         if (round > 2u || (round == 2u && (kept & 1u)))
27             kept = kept + 1u;
28         exp = exp + 1u;
```

```

29     } else {
30         kept = sig >> 1;
31         if ((sig & 1u) && (kept & 1u))
32             kept = kept + 1u;
33         if (kept == 0x1000000u) {
34             kept = 0u;
35             exp = exp + 1u;
36         }
37     }
38     if (exp == 255u)
39         return sign | 0x7f800000u;
40     return sign | (exp << 23) | (kept & 0x7fffffu);
41 }

```

 阶码是 255 时，NaN 和无穷大都原样返回，NaN 的尾数因此保留下来。±0 也原样返回，符号位不动。

非规格化数没有隐含 1。尾数乘 3 再右移 1 位，就是乘 3/2；丢掉的最低位如果是 1，并且留下的最低位也是 1，就再加 1，这是向偶数舍入。结果如果进位到第 23 位，和符号位或在一起就变成最小规格化数。

规格化数先补上隐含的 1 再乘 3。乘积达到 2^{25} 时，数值跨进了下一档阶码：右移 2 位，用低 2 位做向偶舍入，阶码加 1。否则只右移 1 位；若进位把有效数字顶成 2^{24} ，尾数清零、阶码加 1。阶码加到 255，结果就是同号无穷大。

16. floatRoundEven

功能：将 uf 表示的浮点值舍入到最近的整数，恰好处于中间的值舍入到偶数；返回该整数以单精度浮点数表示时的位级形式。若舍入结果为 0 则保留输入符号（负数舍入到 0 时返回 -0）。当 uf 为 NaN 或无穷大时，原样返回 uf。

允许的运算符：任意整数/无符号运算（含 ||、&&），也可使用 if、while

最大运算次数：65 | 难度评级：10

代码块

```

1  unsigned floatRoundEven(unsigned uf) {
2      unsigned sign = uf & 0x80000000u;
3      unsigned exp = (uf >> 23) & 0xffu;
4      unsigned frac = uf & 0x7fffffu;
5      unsigned frac_bits;
6      unsigned half;
7      unsigned mask;
8      unsigned lower;
9      unsigned base;

```

```

10     unsigned lsb;
11
12     if (exp == 255u)
13         return uf;
14     if (exp < 126u)
15         return sign;
16     if (exp == 126u) {
17         if (frac == 0u)
18             return sign;
19         return sign | 0x3f800000u;
20     }
21     if (exp >= 150u)
22         return uf;
23
24     frac_bits = 150u - exp;
25     half = 1u << (frac_bits - 1u);
26     mask = (half << 1) - 1u;
27     lower = frac & mask;
28     base = frac & ~mask;
29     if (frac_bits == 23u)
30         lsb = 1u;
31     else
32         lsb = (frac >> frac_bits) & 1u;
33     if (lower > half || (lower == half && lsb)) {
34         base = base + (half << 1);
35         if (base >> 23) {
36             exp = exp + 1u;
37             base = 0u;
38         }
39     }
40     return sign | (exp << 23) | base;
41 }

```

💡 阶码 255 的 NaN 和无穷大原样返回。阶码小于 126 时绝对值小于 0.5，舍入结果是带符号的 0。阶码等于 126 且尾数为 0 时，数值正好是 0.5，0 和 1 的中点按向偶应舍到 0，同样保留符号；只要尾数不是 0，数值就大于 0.5 且小于 1，更靠近 1。

阶码大于等于 150 时，小数位已经不存在，浮点数本身就是整数。

其余情况下，小数位数是 $150 - \text{exp}$ 。大于一半就进位，小于一半就截断，正好一半时看整数部分的最低位，这一位是 1 才进位。阶码为 127 时，整数部分只剩隐含的 1，它是奇数。进位可能把尾数顶出 23 位，这时阶码加 1，尾数归零。

17. float_i2f

功能：返回表达式 (float) x 的位级等价表示。结果以 unsigned int 返回，但应理解为单精度浮点数的位级表示。

允许的运算符：任意整数/无符号运算（含 ||、&&），也可使用 if、while

最大运算次数：40 | 难度评级：10

代码块

```
1  unsigned float_i2f(int x) {
2      unsigned sign;
3      unsigned absx;
4      unsigned frac;
5      unsigned rest;
6      unsigned exp;
7      int shift;
8
9      if (x == 0)
10         return 0;
11     if (x == 0x80000000)
12         return 0xc0000000u;
13     if (x < 0) {
14         sign = 0x80000000u;
15         absx = -x;
16     } else {
17         sign = 0u;
18         absx = x;
19     }
20     shift = 0;
21     while ((absx & 0x80000000u) == 0u) {
22         absx = absx << 1;
23         shift = shift + 1;
24     }
25     frac = (absx >> 8) & 0x7fffffu;
26     rest = absx & 0xffu;
27     exp = 158u - shift;
28     if (rest > 0x80u || (rest == 0x80u && (frac & 1u))) {
29         frac = frac + 1u;
30         if (frac == 0x800000u) {
31             frac = 0u;
32             exp = exp + 1u;
33         }
34     }
35     return sign | (exp << 23) | frac;
36 }
```



$x = 0$ 时直接返回 0。INT_MIN 取相反数会溢出，而 -2^{31} 本身可以精确表示，位模式是 0xc0000000，单独返回。

其余负数先取出符号，再用 $-x$ 得到绝对值。把绝对值不断左移，直到最高位变成 1，移动次数记为 shift。规格化之后阶码是 $158 - \text{shift}$ ：158 对应最高位原来就在第 31 位时的阶码 $127 + 31$ 。

最高位是隐含的 1，紧接着的 23 位是尾数，再往下的 8 位用来舍入。这 8 位大于 0x80 就进位；等于 0x80 时，尾数最低位是 1 才进位。进位若把尾数加成 0x800000，说明 23 位尾数溢出，尾数清零、阶码加 1。最后把符号、阶码和尾数拼起来。

18. bitCount

功能：返回 x 的二进制表示中 1 的个数。示例：bitCount(5) = 2，bitCount(7) = 3。

允许的运算符： `! ~ & ^ | + << >>`

最大运算次数：40 | 难度评级：10

代码块

```
1  int bitCount(int x) {
2      int m1 = 0x55 | (0x55 << 8);
3      int m2 = 0x33 | (0x33 << 8);
4      int m4 = 0x0f | (0x0f << 8);
5      int m8 = 0xff | (0xff << 16);
6      int m16 = 0xff | (0xff << 8);
7      m1 = m1 | (m1 << 16);
8      m2 = m2 | (m2 << 16);
9      m4 = m4 | (m4 << 16);
10     x = (x & m1) + ((x >> 1) & m1);
11     x = (x & m2) + ((x >> 2) & m2);
12     x = (x & m4) + ((x >> 4) & m4);
13     x = (x & m8) + ((x >> 8) & m8);
14     x = (x & m16) + ((x >> 16) & m16);
15     return x;
16 }
```



用分治统计 1 的个数。0x55、0x33、0x0f 先铺到 16 位，再复制到高 16 位，得到 0x55555555、0x33333333、0x0f0f0f0f。0x00ff00ff 和 0x0000ffff 也用 0xff 拼出来。

每一层都是 $(x \& \text{mask}) + ((x \gg k) \& \text{mask})$ 。先与掩码，再相加，算术右移填进来的符号位会被掩码清掉，不会混进计数值。相邻 1 位合成 2 位计数，再合成 4 位、8 位、16 位，最后高低 16 位相加，低位留下的就是 1 的总数。

19. bitReverse

功能：将 32 位整数 x 的二进制位反转。示例： $\text{bitReverse}(0x80000004) = 0x20000001$ ， $\text{bitReverse}(0x7FFFFFFF) = 0xFFFFFFF$ 。

允许的运算符： `! ~ & ^ | + << >>`

最大运算次数：34 | 难度评级：10

代码块

```
1  int bitReverse(int x) {
2      int m8 = 0xff | (0xff << 16);
3      int m4 = m8 ^ (m8 << 4);
4      int m2 = m4 ^ (m4 << 2);
5      int m1 = m2 ^ (m2 << 1);
6      int m16 = 0xff | (0xff << 8);
7      x = ((x >> 1) & m1) | ((x & m1) << 1);
8      x = ((x >> 2) & m2) | ((x & m2) << 2);
9      x = ((x >> 4) & m4) | ((x & m4) << 4);
10     x = ((x >> 8) & m8) | ((x & m8) << 8);
11     return ((x >> 16) & m16) | (x << 16);
12 }
```

 和 `bitCount` 相反，按块交换位置。先交换相邻 1 位，再交换相邻 2 位、4 位、8 位，最后交换高低 16 位。

每一步都是 $((x \gg k) \& \text{mask}) | ((x \& \text{mask}) \ll k)$ 。掩码如果各自用常数铺开，操作数会超过 34。这里从 `0x00ff00ff` 往回推：它和自己左移 4 位异或得到 `0x0f0f0f0f`，再同样推出 `0x33333333` 和 `0x55555555`。最后一步用 `0x0000ffff` 挡住算术右移补进来的符号位，再和左移 16 位的低半段拼起来。

参考资料

- Randal E. Bryant, David R. O'Hallaron. *Computer Systems: A Programmer's Perspective*. 整数的补码运算，以及 IEEE 754 单精度格式、向偶数舍入。
- `bitCount` 与 `bitReverse` 使用分治的位并行写法：用掩码把相邻位合并或交换，再逐级扩大到 2、4、8、16 位。

Part 3. 测试结果

`./check_ops.py bits.c`

```
root@ics:~/ICS-DataLab# ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 11/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 28/56 operators
rotateRightBits: 15/16 operators
roundEvenPow2: 18/24 operators
midpointTowardFirst: 25/32 operators
isBetweenEitherOrder: 41/48 operators
mul5Sat: 21/30 operators
classifyAdd3: 30/52 operators
floatScaleThreeHalves: 40/60 operators
floatRoundEven: 32/65 operators
float_i2f: 23/40 operators
bitCount: 36/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

./btest

```
root@ics:~/ICS-DataLab# ./btest
Score  Rating  Errors  Function
1      1        0      signMask
2      2        0      bitXor
3      3        0      negativePart
4      4        0      copyByteWithin
4      4        0      logicalShift
4      4        0      swapNibblePairs
4      4        0      secondLowestZeroBit
5      5        0      oddParity
5      5        0      rotateRightBits
5      5        0      roundEvenPow2
5      5        0      midpointTowardFirst
7      7        0      isBetweenEitherOrder
7      7        0      mul5Sat
7      7        0      classifyAdd3
7      7        0      floatScaleThreeHalves
10     10        0      floatRoundEven
10     10        0      float_i2f
10     10        0      bitCount
10     10        0      bitReverse
Total points: 110/110
```

19 道题的运算符检查全部通过，btest 的总分是 110/110。